README.md

Averpo ERP

A double-entry financial ERP for small and medium businesses in Uzbekistan - accounting core, sales, purchasing, multi-warehouse inventory, banking, payroll and IFRS-style financial statements in a single, self-hosted application.

Built in Java 21 / Spring Boot 4 with server-side rendering (JTE + HTMX), no SPA framework, no CDN dependency, and no external service required to run.

uz Ўзбекча (README.uz.md)

Language / Runtime	Java 21, Spring Boot 4.0
Database	PostgreSQL 18, Liquibase (62 migrations, `ddl-auto=validate`)
UI	JTE server-side templates + HTMX + Alpine.js 3, Tailwind CSS 4
Domain size	69 entities · 21 modules · 25 document types
Code	~51 000 lines of production Java, ~27 600 lines of tests
Tests	860 test methods, integration tests on real PostgreSQL
Business rules	253 catalogued rules with unique `BR-*` codes
Languages	Uzbek, Russian, English (1 762 UI keys each)

Table of contents

1. Why this project exists
2. Comparative research: what we studied
3. Accounting standards (IFRS / IAS)
4. Adapting to Uzbekistan
5. Architecture
6. The iron rules
7. Features
8. Screenshots
9. Quality and engineering discipline
10. Getting started
11. Roadmap
12. Documentation
13. Sources and references

1. Why this project exists

Small and medium businesses in Uzbekistan are served badly by the current software landscape:

- **Global cloud products** (QuickBooks Online, Xero) do not localise here: no Uzbek language, no UZS-first workflows, no local banking or tax practice, and pricing in foreign currency per company per month.
- **Large ERPs** (SAP, Oracle NetSuite) are far too expensive and heavy for a 5-50 person company, and require a permanent implementation partner.
- **Spreadsheets** remain the default. They do not enforce double-entry, do not produce a Balance Sheet that provably balances, and leave no audit trail.

Averpo targets the gap in the middle: **the accounting rigour of a real ERP with the footprint and usability of a small-business product**, in Uzbek, in UZS, self-hostable on a single modest server.

---

2. Comparative research: what we studied

Averpo is our own system, designed for our own market - but it was not designed in a vacuum. Before writing the domain model we studied four established systems - **QuickBooks Online, Xero, Oracle NetSuite and SAP S/4HANA** - from their *primary* sources: official XSD schemas, SDKs, API documentation and vendor help systems, not blog posts.

The point was to learn what decades of accounting software have settled on, to see where each of them stops, and then to decide for ourselves. Some of what follows is where we align with them; some is where we deliberately do more.

2.1 QuickBooks Online - validating the domain model

QuickBooks Online is the most widely deployed small-business accounting system in the world, which makes its published data model a useful yardstick for **domain completeness**. We used it to check our own model field by field: is anything essential missing, and does our vocabulary match accepted industry usage?

We worked from Intuit's own artefacts rather than marketing pages:

- `Finance.xsd` and `IntuitNamesTypes.xsd` from the official QuickBooks V3 Java SDK (`ipp-v3-java-data` , Apache 2.0) - vendored into `docs/qbo-reference/` so every claim is checkable.
- A field-by-field comparison of QBO entities against ours in `docs/qbo-reference/entities.md` .

What this gave us - verified directly in the schema:

- The **three-level chart of accounts** (`Classification → AccountType → AccountSubType`), including the exact official `AccountSubType` names.
- The `Transaction` **supertype**: in QBO's own object model every document (`Invoice` , `Bill` , `Payment` , `JournalEntry` , `Transfer` , …) inherits from a common `Transaction` base carrying `DocNumber` , `TxnDate` , `CurrencyRef` , `ExchangeRate` , `Line[]` , `LinkedTxn[]` . Sales documents descend further via `SalesTransaction` , purchase documents via `PurchaseByVendor` .
- The rule that **one document has one currency and one exchange rate** (both live on the header, not the line).

Two areas where we went further, because the businesses we build for need them:

1. **Multi-warehouse inventory** (plus landed cost and a unit-of-measure catalogue) - QBO has no concept of a warehouse at all.
2. **Payroll Lite** - payroll is not part of the QBO core product.

Beyond that we add a feature when the local business case is proven, not because a competitor ships it. Scope discipline is deliberate.

2.2 Xero - verified August 2026

Xero was studied through the Accounting API (OpenAPI v16.1.0) and Xero Central. It is the closest competitor in the SME cloud segment, and the comparison is instructive:

Dimension	Xero	Averpo
Chart of accounts	2 levels: 5 classes → 17 account types	3 levels, QBO-style (`Classification → Type → DetailType`)
Invoice vs Bill	One entity distinguished by `Type` (`ACCREC` / `ACCPAY`)	Separate documents with separate posting rules
Inventory valuation	**Weighted average only** - no FIFO in core	**AVCO or FIFO**, chosen per company, locked after first movement
Warehouses	**None in core.** Xero's own FAQ: "*Xero's core accounting software doesn't track inventory in multiple locations*" - a separate *Inventory Plus* add-on offers it, US-only	Multi-warehouse is a first-class part of the domain
Inventory adjustment	No adjustment document - the documented workaround is a zero-total invoice / credit note	Dedicated adjustment and transfer documents
Posted document edits	An approved, unpaid invoice **can be edited, amounts included**	POSTED is immutable - correction only by reversal
Period close	Lock dates only, and an administrator can remove them at any time	Closing-date lock enforced in the posting service (`BR-LED-020`)
Analytical dimensions	Hard cap of **2 active** tracking categories	Class tracking at line level
Group consolidation	Not in any standard plan (Xero stated in July 2025 that native consolidated reporting was "*not currently planned*"); it arrived only in July 2026 through Syft, bundled into the new Australia-only *Xero Ultra* tier	Multi-tenant design in progress (see roadmap)

2.3 Oracle NetSuite

Studied through the NetSuite Help Center. NetSuite sits at the opposite architectural extreme from QBO:

- **One transaction table for everything.** NetSuite models all document types in a single `transaction` table striped by a `TYPE` column, with lines in `transactionline` and the actual GL debits and credits in a third table, `transactionaccountingline`.
- **Seven costing methods** - Average (default), FIFO, LIFO, Standard, Group Average, Specific, Lot Numbered - with true per-layer costing.
- **Void semantics are configurable.** By default voiding a transaction *mutates the original*, setting its amount to zero. With the *Void Transactions Using Reversing Journals* preference enabled, the original is preserved and a dated reversing journal carries the offset - but enabling it makes invoices, credit memos and cash sales no longer voidable at all.
- **OneWorld / Multi-Book** support multi-subsidiary and parallel accounting books - the closest analogue to our planned multi-tenant model.

What we took: the **discriminator pattern** (used in our `BankTransaction`, which covers deposit / expense / transfer in one table) and the principle that a reversal should land in an *open* period rather than rewriting a closed one.

What we did not take: a single mega-table for all documents. Invoice, payment, purchase order and journal entry have genuinely different shapes and invariants; merging them costs constraint strength and produces very wide, sparse tables.

2.4 SAP S/4HANA

Studied through SAP's official learning material, Help Portal and Knowledge Base articles. SAP contributed the single most influential idea in our architecture:

- **Two layers.** Operational documents live per domain (`VBAK` / `VBAP` sales, `EKKO` / `EKPO` purchasing, `MATDOC` material movements in S/4HANA) and post to accounting through account determination. The accounting document is separate from the operational document.
- **The Universal Journal** (`ACDOCA`, introduced with SAP S/4HANA Finance 1503) unifies FI, CO, asset accounting and material ledger into one line-item table, eliminating reconciliation between sub-ledgers.
- **Posted amounts are immutable.** Corrections go through reversal with a mandatory reason code; only non-value fields (reference, text) can be changed afterwards, governed by document change rules.
- **Perpetual valuation is moving average or standard price.** FIFO and LIFO in SAP are *periodic balance-sheet valuation procedures*, not per-layer perpetual costing - and LIFO is not supported in S/4HANA Cloud at all, because it is not permitted under international standards.

Averpo follows exactly this two-layer philosophy: **documents are operational, the ledger is the single financial truth**, and posted records are immutable.

2.5 Where Averpo lands

	QuickBooks Online	Xero	NetSuite	SAP S/4HANA	Averpo
Document storage	Typed entities under a `Transaction` supertype	One entity per family, type-discriminated	Single `transaction` table	Operational tables per domain	Separate tables, discriminator for bank documents
Unified ledger	Internal	Derived, read-only `Journals`	`transactionaccountingline`	`ACDOCA` Universal Journal	`journal_entry` / `journal_entry_line`
Posted document	Editable	Editable while unpaid	Configurable void	Immutable (reversal)	**Immutable (reversal)**
Inventory valuation	Average (FIFO in Advanced)	Average only	7 methods, per-layer	Moving average / standard	**AVCO or FIFO, per (item, warehouse)**
Multi-warehouse	No	No (core)	Yes	Yes	**Yes**
Target size	Small business	Small business	Mid / large enterprise	Large enterprise	**Small / medium business**

3. Accounting standards (IFRS / IAS)

The chart of accounts and financial statements follow **IFRS presentation**, not a national statutory template. The standards that concretely shaped the code:

Standard	Where it applies
IAS 1 - *Presentation of Financial Statements*	Balance Sheet and Profit & Loss structure; the Balance Sheet asserts `assets = liabilities + equity`, and current-year profit is separated from retained earnings by fiscal year
IAS 2 - *Inventories*	Inventory is measured at cost using **weighted average (AVCO)** or **FIFO** - the two methods IAS 2 permits. **LIFO is not implemented, because IAS 2 prohibits it.** Cost is computed per `(item, warehouse)`
IAS 21 - *The Effects of Changes in Foreign Exchange Rates*	Home (functional) currency is fixed per company and locked after the first posted entry; transactions are recorded at the rate of the transaction date; realised exchange differences on settlement post to profit or loss
IFRS 15 - *Revenue from Contracts with Customers*	Revenue is recognised when the sales document is posted, separated from tax and from cost of goods sold
IFRS 9 - *Financial Instruments*	Receivables and payables as measured balances; AR/AP ageing analysis
IAS 7 - *Statement of Cash Flows*	Cash movement reporting on the dashboard (a full cash-flow statement is on the roadmap)

The double-entry core enforces the accounting equation structurally rather than by convention: **every posting is balanced in home currency (`sum(debitBase) = sum(creditBase)`), asserted by unit tests for every posting rule**, and the general ledger is the only place balances come from - no module keeps its own totals.

This is the model taught by the international accountancy bodies (ACCA, ICAEW) and expected by any IFRS-trained accountant. Standard texts are published by the IFRS Foundation at ifrs.org and are referenced here by number only.

---

4. Adapting to Uzbekistan

Global products are built for other markets. The adaptations that matter here:

- **Uzbek first.** The entire interface is Uzbek (Cyrillic), with Russian and English alongside - 1 762 translated keys per language. Not a partial translation layer: every screen, message and business-rule error.
- **UZS as home currency** by default, with the multi-currency machinery needed for real trade (buying in USD, selling in UZS). Exchange rates are imported **automatically from the Central Bank of Uzbekistan** twice a day, and the system pivots rates correctly whichever currency the company keeps its books in.
- **Number and money formatting for local reading habits**: decimal point, non-breaking-space thousands separator (`12 600.50`), amounts always shown with their currency code, quantities always with their unit.
- **VAT (ҚҚС) as a first-class catalogue** with per-line rate snapshots, so historical documents stay correct when the rate catalogue changes.
- **Payroll Lite** with the local deduction structure (income tax, pension and social contribution rates in company settings) - deliberately outside the QBO reference, because small businesses here expect payroll in the same system.
- **Self-hosted, single-server deployment.** No per-seat cloud subscription in foreign currency, no dependency on an offshore service being reachable.

- **Multi-warehouse inventory**, which trading companies here need and neither QuickBooks Online nor Xero provides in its core product.

---

5. Architecture

5.1 Modular monolith

Not microservices. Packages are organised by business module (`com.averpo.erp.<module>.{domain,service,repo,web}`), and modules talk to each other **only through public service interfaces** - reaching into another module's repository is forbidden. Dependencies all point one way: towards the ledger, and the ledger depends on nobody.

```
sales · purchase · bank · inventory · payroll · pricing · tax · contact · item
                                  ↓
                            ledger (PostingService)
                                  ↓
                      journal_entry / journal_entry_line
```

5.2 The ledger is the single source of truth

No module stores balances. Every balance, every report figure and every dashboard number is computed from the general ledger.

When a document is posted it calls `PostingService` , which creates the journal entry and links it back with `sourceModule` + `sourceDocumentId` . A posted entry is immutable; a mistake is corrected by a reversing entry, never by an update.

5.3 Document lifecycle

```
DRAFT  →  POSTED  →  REVERSED
 free      GL entry    storno entry
 editing   read-only   created
```

Purchase orders and estimates are deliberately GL-free: they participate in the document flow without touching the ledger.

5.4 Persistence

- **UUIDv7 primary keys**, assigned in the constructor (time-ordered, index friendly) rather than generated by Hibernate.
- Money is stored as `NUMERIC(19,4)` , exchange rates as `NUMERIC(24,12)` so that inverse rates (`1 UZS = 0.000082690073 USD`) keep eight significant digits.
- **Liquibase is the only schema authority** - 62 sequential migrations; Hibernate runs in `validate` mode and never generates DDL.
- All timestamps are stored in UTC and rendered in the company's timezone.

---

6. The iron rules

Thirteen invariants that every code review checks. The full text is in `docs/engineering-rules.md` :

1. Money is a `Money` value object - never `double` or `float`.
2. Only `PostingService` writes to the general ledger.
3. A POSTED document is never modified - reversal only.
4. The ledger balances in home currency: `sum(debitBase) = sum(creditBase)`.
5. Every schema change is a new Liquibase changeset.
6. Cross-module access goes through service interfaces only.
7. Every posting rule has a unit test asserting debit = credit.
8. Postings must match `docs/posting-rules.md` exactly.
9. Inventory valuation (AVCO/FIFO) is chosen per company and locked after the first stock movement.
10. Every field and method carries documentation explaining *why*.
11. Currency is a catalogue entity; `Money` stores the ISO code.
12. All times are stored in UTC.
13. Business-rule violations raise `BusinessRuleException` with a unique `BR-*` code from `docs/business-rules.md`.

---

7. Features

Accounting core - IFRS-style chart of accounts (three levels, QBO taxonomy), manual journal entries, opening balances, closing-date lock, audit log.

Sales - Invoice, Sales Receipt, Estimate, Credit Memo, Refund Receipt, customer payments with allocation across invoices, AR ageing, credit limits, price lists with quantity tiers.

Purchasing - Bill, Purchase Order (GL-free), Vendor Credit, vendor payments with allocation, AP ageing, **landed cost** allocation onto received goods.

Inventory - multi-warehouse stock, AVCO or FIFO valuation per `(item, warehouse)`, receipts, issues, adjustments, transfers, unit-of-measure groups with conversion factors, inventory valuation report as of any date.

Banking - bank accounts, deposits, expenses, transfers, currency conversion with automatic exchange gain/loss, reconciliation.

Payroll Lite - employees, salary rates, payroll runs, partial payments, payroll register.

Reporting - Balance Sheet, Profit & Loss, Trial Balance, P&L by class, AR/AP ageing with drill-down, inventory valuation, customer statements, and a QBO-style dashboard.

Platform - 8 roles with area-based permissions, audit trail, attachments, global search, Excel import for opening data, three languages, light and dark themes, mobile-first layouts (every screen works at 375 px).

---

8. Screenshots

Screenshots of the running application are collected in `docs/screenshots/`.

---

9. Quality and engineering discipline

- **860 test methods**, the majority of them integration tests running against a real PostgreSQL database - not mocks, not H2.
- **Every posting rule is tested for balance**: debit equals credit, in home currency, for every document type and every reversal path.
- **253 business rules** catalogued with unique codes before they are implemented. A rule that is not in the catalogue does not exist in the code.
- **Specification before code**: each module has a written specification in `docs/modules/` agreed before implementation starts.
- **Schema is versioned, never generated** - Hibernate validates, Liquibase owns.
- **Documentation is part of the deliverable**: 50+ living documents covering architecture, posting rules, business rules, UI conventions and every module.

---

10. Getting started

Requirements: JDK 21, PostgreSQL 18.

```
# 1. Create the database and role
psql -U postgres -c "CREATE ROLE averpo LOGIN PASSWORD 'averpo';"
psql -U postgres -c "CREATE DATABASE averpo OWNER averpo;"

# 2. Run - Liquibase builds the schema and a default chart of accounts is seeded
./gradlew bootRun --args='--spring.profiles.active=dev'
```

Open http://localhost:8080 and sign in as `admin`. In `dev` the password is seeded automatically; in production `AVERPO_ADMIN_PASSWORD` is **mandatory** - the application refuses to start without it.

Sample data. Start with the `demo` profile to populate a fictional trading company - customers, vendors, items, posted invoices, bills and payments - so that reports and the dashboard show meaningful figures:

```
./gradlew bootRun --args='--spring.profiles.active=dev,demo'
```

Tests (require a database whose name ends in `_test` - a safety guard refuses to run against anything else):

```
psql -U postgres -c "CREATE DATABASE averpo_test OWNER averpo;"
./gradlew test
```

---

11. Roadmap

Multi-tenant SaaS. The next major phase turns Averpo into a multi-company platform: one PostgreSQL database with a `tenant_id` discriminator, Hibernate native `@TenantId` filtering, PostgreSQL row-level security as a second defence layer, and session-based tenant resolution that leaves existing URLs untouched. The full plan is in `docs/multi-tenant-plan/plan-for-averpo.md`.

Accountant portal. One accountant, many companies - the model outsourced bookkeeping in Uzbekistan actually runs on: a single login managing the accountant's own firm books plus every client company, with per-client access for staff.

Also planned: recurring transactions, document printing / PDF / email, bank statement import with matching, VAT period reporting, and budgeting.

---

12. Documentation

The `docs/` tree is the project's working memory, kept current with the code:

Document	Contents
`architecture.md`	Architectural decisions and module structure
`engineering-rules.md`	The iron rules and code conventions
`posting-rules.md`	The exact GL entries for every document type
`business-rules.md`	All 253 `BR-*` rules
`modules/`	One specification per module
`qbo-reference/`	Official QuickBooks XSD schemas and field-level comparison
`multi-tenant-plan/`	The multi-tenant migration plan
`ui-style-guide.md`	Screen patterns, colours, form and table conventions

A print-ready PDF of the complete documentation set is generated into `docs-pdf/`.

---

13. Sources and references

QuickBooks Online - QuickBooks Online API documentation - QuickBooks V3 Java SDK - `Finance.xsd`, `IntuitNamesTypes.xsd` (vendored in `docs/qbo-reference/`) - QuickBooks Desktop API reference - reviewed for completeness; desktop-only (`QBW`) fields were excluded as legacy and irrelevant to a modern cloud data model

Xero - Xero Accounting API overview - Xero Central - inventory, multicurrency, lock dates, tracking categories

Oracle NetSuite - NetSuite Help Center

SAP - SAP Help Portal and SAP Learning - Universal Journal, document reversal, material valuation

Accounting standards - IFRS Foundation - list of standards (IAS 1, IAS 2, IAS 7, IAS 21, IFRS 9, IFRS 15) - ACCA - the professional practice the design follows

Central Bank of Uzbekistan - Daily exchange rate service, consumed by the automatic rate import

---